

# LABORATORY OBSERVATION

**Student Name:** P. Dharani

**Course:** Full Stack Web

Development

**Experiment No.:** 5-2

---

## 1. TITLE

Node.js File Management

---

## 2. AIM

To create a Node.js application that accepts a filename and content from the user and performs file creation/writing, reading, appending, and final content display using the Node.js File System module.

---

## 3. OBJECTIVE

The objectives of this experiment are:

- To understand the Node.js File System (fs) module.
  - To create and write data to a file.
  - To read existing file contents.
  - To append additional content to a file.
  - To display the final contents after file operations.
  - To handle invalid input and runtime errors.
- 

## 4. THEORY

Node.js provides built-in modules for performing server-side operations. The File System (fs) module provides methods for creating, reading, writing, and modifying files.

This application uses `writeFileSync()` to create a file and write the initial content, `readFileSync()` to read the file, and `appendFileSync()` to add additional content to the existing file.

The `readline` module is used to accept filename and content from the user through the terminal. A Promise-based helper function is used so that the input operations can be performed sequentially.

---

## 5. ALGORITHM / PROCEDURE

1. Import the File System (fs) and readline modules.
  2. Create a readline interface for terminal input.
  3. Ask the user to enter a filename.
  4. Ask the user to enter the initial content.
  5. Validate that the filename is not empty.
  6. Create the file and write the initial content using `writeFileSync()`.
  7. Read and display the contents using `readFileSync()`.
  8. Ask the user for additional content.
  9. Append the additional content using `appendFileSync()`.
  10. Read and display the final file contents.
  11. Handle errors using try-catch and close the readline interface.
-

## 6. PROGRAM

### app.js

```
const fs = require("fs");
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function ask(prompt) {
  return new Promise((resolve) => {
    rl.question(prompt, resolve);
  });
}

async function manageFile() {
  try {
    const filename = (await ask("Enter filename: ")).trim();

    if (filename === "") {
      console.log("Error: Filename cannot be empty.");
      return;
    }

    const initialText = await ask("Enter content to write: ");

    // Step 1: create the file and write the initial content
    fs.writeFileSync(filename, initialText);
    console.log("\nFile created and content written successfully.");

    // Step 2: read the file and display its contents
    console.log("\nFile contents after writing:");
    console.log(fs.readFileSync(filename, "utf8"));

    // Step 3: append more content on a new line
    const extraText = await ask("Enter additional content to append: ");
    fs.appendFileSync(filename, "\n" + extraText);
    console.log("\nAdditional content appended successfully.");

    // Step 4: read the file again and display the final contents
    console.log("\nFinal file contents:");
    console.log(fs.readFileSync(filename, "utf8"));
  } catch (error) {
    console.log("Error: " + error.message);
  } finally {
    rl.close();
  }
}

manageFile();
```

### package.json

```
{
  "name": "q2-nodejs-file-management",
  "version": "1.0.0",
  "description": "Node.js file management using the File System module",
  "main": "src/app.js",
  "scripts": {
    "start": "node src/app.js"
  },
  "author": "D.vijaya sri",
  "license": "ISC"
}
```

---

## 7. CODE EXPLANATION

The program asks the user for a filename and some text, and then performs the four file operations in order: it creates the file with the text, reads it back, appends a second line of text, and reads the file again to show the final content. **ask()** wraps `rl.question()` in a Promise so that each prompt can be awaited one after another inside the async function **manageFile()**. The variables **filename**, **initialText** and **extraText** hold the user's inputs. If the filename is empty the program prints an error and stops; any error thrown by an fs method (for example, a folder that does not exist) is caught by the try...catch block and printed. The **finally** block always closes the readline interface. **package.json** describes the project and lets it be started with npm start.

| Code                                                      | Explanation                                                                                   |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>require("fs")</code>                                | Imports the built-in Node.js File System module.                                              |
| <code>require("readline")</code>                          | Imports the module used to accept input from the terminal.                                    |
| <code>readline.createInterface()</code>                   | Creates an interface for reading user input from stdin and writing prompts to stdout.         |
| <code>ask()</code>                                        | Returns user input as a Promise so operations can run sequentially using <code>await</code> . |
| async function <code>manageFile()</code>                  | Contains the complete workflow; async allows the use of <code>await</code> for each prompt.   |
| <code>trim()</code> and <code>if (filename === "")</code> | Removes extra spaces and stops with an error message if no filename was entered.              |
| <code>writeFileSync()</code>                              | Creates the file and writes the initial content.                                              |
| <code>readFileSync()</code>                               | Reads and returns the file contents (used after writing and again after appending).           |
| <code>appendFileSync()</code>                             | Adds additional content (on a new line) to the end of the existing file.                      |
| <code>try...catch</code>                                  | Handles errors that may occur during file operations and prints the error message.            |
| <code>rl.close()</code>                                   | Closes the readline interface after execution (in the finally block).                         |

## 8. TEST CASES AND EXECUTION DATA

| Test Case | Input                                                                                                | Expected Result                                                              | Actual Result                                                            | Status |
|-----------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------|--------|
| TC-01     | Filename: notes.txt<br>Content: Node.js is a JavaScript runtime.<br>Append: It uses the V8 engine.   | File is created, read, appended, and final content (two lines) is displayed. | File created, read, appended; final content shows both lines.            | Pass   |
| TC-02     | Empty filename                                                                                       | An appropriate filename error is displayed.                                  | Error: Filename cannot be empty. (program stops; no file created)        | Pass   |
| TC-03     | Filename: report.txt<br>Content: Lab report for Experiment 5-2<br>Append: Append operation completed | Additional content is appended to the existing file.                         | Append operation completed added on a new line after the original text.  | Pass   |
| TC-04     | Filename: missing_folder/data.txt (folder does not exist)                                            | The error should be caught and displayed.                                    | Error: ENOENT: no such file or directory, open 'missing_folder/data.txt' | Pass   |

## 9. OUTPUT

### Output 1: Terminal Execution of Node.js File Management (TC-01)

```
C:\Q2_NodeJS_File_Management\src>node app.js
Enter filename: notes.txt
Enter content to write: Node.js is a JavaScript runtime.

File created and content written successfully.

File contents after writing:
Node.js is a JavaScript runtime.
Enter additional content to append: It uses the V8 engine.

Additional content appended successfully.

Final file contents:
Node.js is a JavaScript runtime.
It uses the V8 engine.

C:\Q2_NodeJS_File_Management\src>
```

The screenshot shows successful file creation and writing, reading of the initial contents, appending of additional content, and display of the final contents.

### Output 2: Empty Filename Validation (TC-02)

```
C:\Q2_NodeJS_File_Management\src>node app.js
Enter filename:
Error: Filename cannot be empty.

C:\Q2_NodeJS_File_Management\src>
```

### Output 3: File Operation Error Handled by try...catch (TC-04)

```
C:\Q2_NodeJS_File_Management\src>node app.js
Enter filename: missing_folder/data.txt
Enter content to write: This will fail
Error: ENOENT: no such file or directory, open 'missing_folder/data.txt'

C:\Q2_NodeJS_File_Management\src>
```

## 10. RESULT

Thus, the Node.js File Management application was implemented using the File System module. The execution output verifies successful file creation, writing, reading, appending, and display of the final file contents, and shows that an empty filename and a failed file operation are reported with error messages.

## How this would be submitted

### Source Code

```
Q2_NodeJS_File_Management/
├── src/
│   └── app.js
├── package.json
└── README.md
```